SDG BLOCKCHAIN ACCELERATOR

Integration Report: Cohort 2 Pilot Projects

1. Executive Summary

The second cohort of pilot projects under the EMURGO–UNDP accelerator program builds directly on the technical foundations established during Cohort 1. While Cohort 1 primarily focused on onboarding teams to Cardano-based architectures and validating core SDG-aligned use cases, Cohort 2 reflects a **shift toward system integration, interoperability, and deployment readiness**.

This report documents the **technical integration, architectural alignment, and validation activities** conducted during Cohort 2, with emphasis on:

1. Reuse and extension of EMURGO-developed reference repositories and scaffolds.
2. Integration of blockchain infrastructure into **existing production systems**.
3. Cross-team interoperability (e.g., wallet infrastructure reused across solutions).
4. Expansion toward **offline-first access**, **digital identity**, and **regulatory-aligned reporting**.

All activities were carried out in alignment with UNDP accelerator objectives, ensuring Cohort 2 projects produced **functional, testable, and extensible prototypes** with a clear pathway to real-world deployment.

2. Introduction

2.1 Background

Blockchain-enabled SDG solutions increasingly require more than isolated proof-of-concepts. Real-world deployment contexts, particularly in emerging markets, demand:

- Interoperable systems spanning identity, payments, and data verification.
- Support for users without smartphones, internet access, or banking.
- Alignment with regulatory, institutional, and donor reporting requirements.

Building on insights from Cohort 1, EMURGO Labs continued to provide **modular technical building blocks** that allow teams to focus on domain-specific innovation rather than foundational infrastructure.

2.2 Objective of the Report

The objectives of this report are to:

- Catalog the EMURGO reference repositories and scaffolds applicable to Cohort 2.
- Map Cohort 2 pilot projects to the technical resources used or recommended.

- Document integration methodologies, including deviations for mature platforms.
- Capture architectural insights and lessons learned to inform future cohorts.

3. EMURGO Technical Reference Resources

3.1 Overview of Repositories and Scaffolds

EMURGO Labs provided the same core technical stack introduced in Cohort 1, ensuring continuity and reducing onboarding friction.

No.	Group	Repository	Description	Integration Purpose
1	Aiken Smart Contracts	hello-world-aiken-smart-contract	Basic Aiken contract demonstrating deployment and interaction	Onboarding, transaction lifecycle understanding
2	Aiken Smart Contracts	crowdfunding-aiken-smart-contract	Pooled fund logic with pledge and refund mechanisms	Aid disbursement, grants, climate finance
3	Aiken Smart Contracts	aiken-unit-test	Unit testing framework for validators	Contract correctness and regression testing
4	Plutus Smart Contracts	nft-ticketing-plutus-smart-contract	NFT minting and lifecycle management	Traceability, credentials, impact tokens
5	Plutus Smart Contracts	credit-scoring-plutus-smart-contract	Tokenized scoring and eligibility model	Access control, reputation, conditional flows
6	Plutus Smart Contracts	lending-plutus-smart-contract	Lending protocol using scoring tokens	SME finance, asset-backed lending
7	Backend (NestJS)	template-backend-nestjs	Scalable API backend boilerplate	Off-chain orchestration, integrations
8	Backend (NestJS)	wallet-management-backend-nestjs	Wallet handling and signing service	Custody abstraction and UX simplification

9	Cardano DApp	cardano-dapp-template-nextjs	Front-end scaffold with wallet connectors	User-facing interaction and testing
10	Tokenization APIs	API-Carbon-credit-tokenization	API-based asset tokenization	Carbon and impact asset lifecycle

3.2 Rationale for Continued Use

The reuse of these repositories across Cohort 2 ensured:

- Consistency in on-chain patterns.
- Faster security reviews and validation.
- Easier cross-team interoperability.
- Reduced technical debt for UNDP-aligned deployments.

4. Cohort 2 Pilot Projects Integration

4.1 Project-to-Repository Mapping

No.	Project	Geographic Focus	Focus Area	Reference Repositories Applied	Integration Summary
1	xCapit	Latin America & Africa	Humanitarian Aid, Financial Inclusion, Low-Tech Wallet Infrastructure	Custom Aiken Time-Locked Aid Smart Contracts, wallet-management-backend patterns (adapted), Cardano DApp patterns, SMS/USSD integration	decentralized humanitarian aid distribution system on Cardano using a Low-Tech Wallet (LTW) model. Aid funds are locked in Aiken smart contracts with time-bound conditions and beneficiary constraints. Beneficiaries access funds via SMS on feature phones, while merchants complete transactions through a web platform, ensuring transparent, fraud-resistant aid delivery to unbanked populations.
2	ClimaFi	Global	Climate Finance, Data Verification, Results-Based Financing	API-Carbon-credit-tokenization (adapted), nft-ticketing-plutus-smart-contract	ClimaFi implements an off-chain data ingestion and verification layer that validates real-world climate datasets against predefined standards (e.g., GBON/WIS2 or MRV criteria). A

				(metadata patterns), template-backend-nestjs (adapted)	rules-based “Verification Engine” ensures only compliant data is processed and linked to on-chain actions. Verified datasets trigger minting of “impact” or “data receipt” tokens on Cardano, establishing a cryptographic link between real-world data and blockchain state. The system also supports “shadow accounting” models, enabling simulation of results-based payments prior to live financial settlement, effectively acting as a verification oracle bridging data and on-chain financial logic.
3	Cubid Protocol	Africa & Global South	Energy Settlement, Carbon Accounting, Rural Microgrids	Custom Hydra-side Validators, Custom Aiken L1 Minting Policies, Hydra Runtime, Cardano Node / CLI Tooling, Blockfrost Integration, Frontend Dashboard Repositories	Implemented a decentralized energy settlement and carbon accounting prototype using Cardano Hydra for high-frequency operational accounting and Cardano Layer 1 for low-frequency carbon asset issuance. The system supports custodial consumer balances, internal NGN-denominated accounting credits, deterministic accumulation of delivered energy, and periodic issuance of fungible carbon credits and certificate NFTs on Cardano .
4	Plastiks	LATAM, India, Global	Circular economy	N/A (existing platform)	Mature platform did not reuse scaffolds directly; new integration introduced via xCapit USSD wallet infrastructure.
5	Fuel Switch	Europe (Western Balkans)	Renewable Energy Certification & CBAM Compliance	Custom Aiken NFT Mint/Burn Contract, template-backend-nestjs (adapted), Cardano DApp patterns,	Implemented an Aiken-based smart contract to mint and retire Renewable Energy Certificates (RECs) as NFTs on Cardano. Each NFT represents up to 1000 kWh of verified renewable energy. Off-chain backend orchestrates minting, validation, and

				Lucid-evolution SDK	burn flows via API, anchoring auditable production data on-chain to support CBAM-aligned electricity export reporting.
6	TradeChain X	Africa (Ethiopia)	Carbon Markets, MRV, Tokenized Commodities	Custom Aiken MRV Smart Contracts, template-backend-nestjs (adapted), API-Carbon-credit-tokenization patterns, Cardano DApp patterns	Implemented a blockchain-based MRV system recording the full carbon credit lifecycle on Cardano. Aiken smart contracts manage project registration, verification state transitions, and carbon credit minting. Off-chain services orchestrate verification workflows, marketplace interactions, and retirement logic, reducing fraud and double counting in carbon markets.
7	SALA	UNDP Bosnia & Herzegovina / University of Sarajevo	Digital Identity, Credential Verification, Education	nft-ticketing-plutus-smart-contract, Aiken Credential Anchoring Smart Contracts, Cardano DApp patterns, off-chain storage integration	implemented a decentralized academic credential verification system on Cardano. Educational institutions issue digitally verifiable certificates whose hashes are anchored on-chain via Aiken smart contracts. Third parties verify authenticity through QR codes and blockchain proofs, ensuring tamper resistance, integrity, and privacy-by-design compliance.
8	Green Giraffe	Angola & Zambia	Agri-Tech, Supply-Chain Traceability, SDG Impact Reporting	nft-ticketing-plutus-smart-contract, template-backend-nestjs, custom Aiken Traceability Smart Contracts, Cardano CLI deployment scripts, Cardano DApp patterns	Aiken-based traceability system on Cardano enabling minting, transfer, and verification of agricultural batch tokens. Each token represents a verifiable unit of farm produce, anchoring supply-chain events and impact data on-chain to improve transparency, trust, and market access for producers.

4.2 Detailed Integration Notes

Plastiks (Cohort 2 Extension)

- Plastiks had already developed a Cardano-based PoC during Cohort 1.
- As a result, EMURGO scaffolds were **not reused directly**.
- **New integration introduced:** xCapit's USSD-based wallet infrastructure.
- **New user type enabled:** informal trash collectors without smartphones.
- This represents a **cross-cohort, cross-team integration**, facilitated by EMURGO architecture alignment.

SALA (Identity Stack Evolution)

- Initially integrated with **iden3** for credential verification.
- EMURGO supported architectural evaluation for migration to **Veridian**, a Cardano-native DID framework.
- Planned pilot with **UNDP Bosnia and the University of Sarajevo**, leveraging NFT-based credentials anchored on Cardano.

5. Complementary Tools and Integrations (Cohort 2)

Tool / Integration	Function	Used By	Purpose / Outcome
Aiken CLI (pinned per project) + aikup	Smart contract build/test + version management	Most projects	Compile validators, run unit tests, and maintain reproducible builds through pinned versions and version-manager support.
Lucid Evolution (`@evolution-sdk/lucid`)	Off-chain transaction builder / submission	Fuel Switch, TradeChainX	Constructs, signs, and submits mint, burn, and lifecycle transactions for Aiken / Plutus validators.
Mesh SDK (`@meshsdk/core`)	Wallet and transaction handling	xCapit	Supports wallet connectivity and transaction flows across LTW services and merchant platform interactions.

Blockfrost API	Hosted Cardano API access	xCapit, Fuel Switch, SolarVillage, TradeChainX, SALA	Used for UTxO queries, transaction submission, blockchain monitoring, and application-level chain integration without operating full node infrastructure in all environments.
Cardano CLI / cardano-node	Low-level transaction build/sign/submit and node interaction	Green Giraffe, SALA, SolarVillage	Used for deterministic deployment flows, transaction signing, local node interaction, and operational blockchain validation on testnet.
Hydra Node	High-frequency off-chain settlement	SolarVillage	Supports low-cost, high-frequency operational accounting for energy settlement inside Hydra heads before anchoring key state transitions to Cardano L1.
etcd	Hydra coordination and runtime state support	SolarVillage	Used to coordinate Hydra head operations and runtime state management in the local technical environment.
Mithril Client	Fast bootstrap of Cardano chain state	SolarVillage	Accelerates node synchronization and environment setup for Hydra and Cardano runtime testing.
Docker / Docker Compose	Infrastructure orchestration	SolarVillage, xCapit	Used to run Cardano node, Hydra services, Mithril client, and other support infrastructure in a reproducible development environment.
websocat	Hydra WebSocket interaction	SolarVillage	Used for direct interaction with Hydra head APIs during initialization, commits, snapshots, and decommit flows.

IPFS pinning (e.g. Pinata)	Off-chain document storage	TradeChainX, SALA, optionally ClimaFi	Stores MRV evidence, certificates, and supporting documents while hashes or references are anchored on-chain.
Supabase	Off-chain DB + auth	TradeChainX	Stores project metadata, users, and verification records linked to on-chain state changes.
Redis	Idempotency / concurrency guard	Fuel Switch, xCapit	Prevents duplicate processing during minting, settlement, or retry-heavy backend workflows.
SMS providers (Twilio / KotaniPay)	Feature-phone access	xCapit	Enables OTP flows, balance checks, and beneficiary interactions through low-connectivity channels.
Next.js / React	Web UIs	xCapit Merchant Platform, SALA sandbox UI, SolarVillage frontend	Supports administrative dashboards, verification portals, and operational management interfaces.
pnpm / Vercel-ready frontend tooling	Frontend dependency management and deployment	SolarVillage	Supports frontend build, linting, testing, and deployment of dashboard and landing page repositories.

6. Technical Engagement, Challenges, and Outcomes

During Cohort 2, EMURGO Labs supported each Solution Maker through architecture reviews, contract design sessions, integration troubleshooting, and deployment validation. This section documents per-team technical engagement and verified prototype outcomes, based on the project PoC documentation and demonstrated testnet/sandbox results where available.

6.1 xCapit

Technical Engagement

- Reviewed end-to-end architecture for Low-Tech Wallet (LTW) humanitarian flows, focusing on UTxO contention, time-locked claims, and recovery semantics.
  Smart contract reviews on redeemer design and signature requirements (Admin/NGO, Beneficiary, Steward).
- Backend/API review sessions for merchant order lifecycle (create, OTP verify, settle), including JWT gating and operational endpoints.
- Deployment readiness checks for Preview testnet execution and key material handling (xprv, funded steward wallet, UTxO distribution).

Key Technical Challenges

- Ensuring beneficiary-only consumption of AID UTxOs, despite SMS-based UX abstraction.
- Enforcing ***claim_start/claim_end*** constraints and preventing premature or expired consumption.
- Implementing safe recovery of unclaimed funds by Steward after ***claim_end.***
  Avoiding transaction contention when assigning multiple beneficiaries using the same steward key/wallet.

Solutions Implemented (as reflected in PoC)

- Aiken validator enforces:
  - Beneficiary signature required for “Consume AID”
  - Time-lock checks for ***claim_start*** and ***claim_end***
  - Steward signature required for “Recover AID”
- Backend service provides JWT-authenticated endpoints for:
  - beneficiary registration and lifecycle operations
  - order creation and completion via OTP
  - balance queries and order status checks
- Operational guidance included distributing multiple UTxOs in steward wallet to reduce contention.

Verified Technical Outcomes (from reported results)

- Preview testnet execution confirmed:
  - **Assign AID** transactions succeed (tx hash provided)
  - **Consume AID** and **Recover AID** validate within execution limits (CPU/Memory reported)
- Negative-path validator behavior confirmed:

    - Consume without beneficiary signature rejected
    - Consume before ***claim_start*** rejected
    - Consume after ***claim_end*** rejected
    - Recover without steward signature rejected
    - Recover before ***claim_end*** rejected

6.2 Fuel Switch

Technical Engagement

- Validator design review for REC lifecycle enforcement (mint constraints + burn path).
- Off-chain orchestration sessions on policy derivation, blueprint usage, API surface (/mint), and metadata enforcement.
- Reliability sessions on idempotency and transaction submission (Redis + Blockfrost). Review of unit-test scenarios to ensure mint/burn cannot be abused (e.g., burn redeemer cannot mint).

Key Technical Challenges

- Enforcing strict minting authorization (admin-only) at contract level.
- Preventing issuance of RECs above the permitted granularity limit (≤1000 kWh per NFT).
- Ensuring asset name and metadata format consistency to support transparent auditing.
- Ensuring burn flow cannot be exploited to mint or increase supply.

Solutions Implemented (as reflected in PoC)

- Aiken validator checks:
    - Admin signature required for mint
    - Energy cap enforced (rejects >1000 kWh)
    - Asset name must match metadata schema (redeemer-linked)
    - Burn action rejects any positive mint attempt

- Off-chain backend (Node/Express) integrates:
    - Lucid Evolution for tx building/signing/submission
    - Redis idempotency layer to prevent duplicate mint processing
    - Blockfrost for Preprod network access

Verified Technical Outcomes (from reported results)

- Unit tests executed with 8+ scenarios (as stated), including:

    - reject >1000 kWh tokens
    - reject name mismatch
    - reject burn exploitation attempts

- Preprod transaction outcomes reported:
    - Mint batch success (15 NFTs)
    - Burn REC success (retirement)
    - Unauthorized mint (no admin) rejected

6.3 TradeChainX (Blockchain-Based Carbon Credits MRV Platform)

Technical Engagement

- Contract architecture sessions around multi-validator separation (registry, verification, minting policy).
- Lifecycle modeling workshops (project registration, verification, mint readiness, token mint, retirement).
- Off-chain integration planning for linking on-chain state transitions with Supabase-managed metadata and UI workflows.
- Review of test cases for invalid signatures, unauthorized minting, and invalid state transitions.

Key Technical Challenges

- Representing project lifecycle state transitions safely in a UTxO model.
- Preventing duplicate mint attempts and unauthorized issuance.
- Linking evidence (MRV docs) without exposing sensitive content on-chain.
- Designing retirement semantics to prevent double counting after purchase.

Solutions Implemented (as reflected in PoC)

- Aiken contracts separated by responsibility:
    - project registry validator
    - verification validator
    - minting policy

- Off-chain stack defined:
    - Lucid-based scripts for deployment and transaction flows
    - Node.js backend services
    - Supabase for project/user metadata and coordination

    - IPFS pinning (Pinata) for MRV document storage with on-chain anchoring via hashes

Verified Technical Outcomes (from provided doc)

- PoC includes unit testing + emulator testing plan with expected rejection cases:
    - invalid signatures rejected
    - unauthorized minting fails
    - invalid verifier address rejected
    - incorrect state transitions rejected
    - duplicate mint attempts covered as an edge case

- Deployment approach defined for multiple validators and minting policy on Preprod.

6.4 SALA (Digital Identity & Credential Verification)

Technical Engagement

- Architecture review sessions focused on privacy-by-design: on-chain anchoring of hashes vs off-chain document storage.
- Verification flow review for QR-based public verification (read-only, no wallet required).
- Repository structure alignment and contract integration planning (API ↔ contract linkage).
- Discussion of identity layer options and VC alignment path (W3C VC, Veridian consideration).

Key Technical Challenges

- Preventing credential tampering while avoiding storage of personal data on-chain.
- Designing a public verification UX that is verifiable yet simple (QR flow).
- Linking off-chain certificate documents (PDF/images) to immutable on-chain anchors.
- Completing API-to-contract wiring and increasing test coverage.

Solutions Implemented (as reflected in PoC)

- Credential integrity approach defined:
    - hash anchoring on Cardano via Aiken contracts
    - off-chain storage of documents
    - QR verification flow to resolve and validate anchors

- Sandbox environment provided for demonstration:

    - public verification UI
    - blockchain anchor confirmation (as described)

Verified Technical Outcomes (from provided doc)

- Live sandbox URL provided for demonstration.
- Defined next steps explicitly include:
    - finalizing digital identity integration / VC alignment
    - completing API-to-contract integration
    - migrating Preview to Preprod
    - finalizing IPFS storage integration
    - performance/load testing

6.5 Green Giraffe (Digital Traceability & Market Access Pilot)

Technical Engagement

- Validator logic review for authorization rules (who can mint/transfer/update traceability tokens).
- Testing sessions covering datum/redeemer validation and negative-path enforcement.
- Deployment walkthrough using cardano-cli build/sign/submit flows on Preprod.
- Performance review based on reported CPU/memory usage per transaction.

Key Technical Challenges

- Enforcing permitted participant actions (authorization) for token lifecycle events.
- Preventing malformed datum structures and invalid redeemers.
- Ensuring double-spend prevention through correct UTxO consumption patterns.
- Preparing for pilot scale-up (dashboarding, oracle inputs, audit readiness).

Solutions Implemented (as reflected in PoC)

- Aiken validator scripts enforce:
    - authorization checks for mint/update/transfer
    - datum/redeemer structure validation
    - rejection of invalid redeemers and unauthorized signatures

- Test approach includes:
    - unit tests (`aiken test`)
    - emulator integration tests (`aiken emu`)

Verified Technical Outcomes (from reported results)

- Preprod testnet results reported:
  - Mint batch token success (CPU/memory reported)
  - Transfer token success (CPU/memory reported)
  - Invalid redeemer correctly rejected

- Walkthrough video link provided demonstrating mint, transfer, and verification flow.

6.6 Plastiks

Technical Engagement

- Cross-team integration discussions with xCapit to enable low-connectivity payment flows for a new user type (trash collectors).
- Interface alignment sessions to map Plastiks operational workflow to USSD/SMS wallet interactions.

Key Technical Challenges

- Introducing payments/settlement access for feature-phone users.
- Integrating external wallet infrastructure without re-architecting Plastiks' existing platform.
- Ensuring new user type support does not break existing traceability or accounting flows.

Solutions Implemented

- Plastiks did not adopt EMURGO scaffolds due to existing implementation maturity.
- Integration work focused on enabling xCapit USSD wallet capability for trash collector payments.

6.7 SolarVillage

Technical Engagement

The technical engagement for SolarVillage focused on a very different problem from the rest of the cohort. Rather than treating Cardano primarily as a settlement layer or asset issuance environment, the project was designed around a two-layer architecture: **Hydra for high-frequency micro-transactions** and **Cardano Layer 1 for lower-frequency, higher-assurance carbon issuance and anchoring**.

Support sessions therefore concentrated on three main areas. The first was architectural separation between operational accounting and Layer 1 settlement. The second was runtime and infrastructure setup, particularly around Hydra node operation, commits, decommits, and interaction patterns. The third was around identifying what would need to change for the project to move from a technically valid PoC to something operationally reliable enough for a pilot.

Particular attention was given to:

- Hydra head initialization and transaction lifecycle
- separation between custodial internal balances and L1-issued carbon assets
- reserve wallet setup and asset commit / decommit flows
- operational complexity introduced by manual scripts and infrastructure dependencies
- security and trust assumptions, especially around operator custody and wallet management

Key Technical Challenges

The main challenge in SolarVillage was not whether the concept made sense — it did — but whether the current implementation was operationally manageable.

Several issues came up repeatedly during review:

- Hydra setup was still relatively manual and required multiple moving parts to be configured correctly
  environment setup depended on command-line exports and scripted steps that increase the chance of operator error
- wallet and signing key handling had not yet been formalized to a level appropriate for pilot deployment
- Hydra validation had only been demonstrated in a single-node context, which is useful for proving the flow but not enough for validating real multi-party settlement assumptions
- the head lifecycle remained too dependent on manual WebSocket / curl interactions rather than automated orchestration
- the architecture still needed stronger failure recovery procedures, especially for restarts, node failures, and coordination-layer interruptions

There was also a broader architectural challenge: the system introduces a meaningful amount of operational complexity in exchange for settlement speed and cost reduction. That trade-off may be justified, but only if the reliability layer around Hydra is made much stronger.

Solutions Implemented

The PoC did demonstrate several important technical solutions already in place.

Hydra was used to handle high-frequency internal settlement activity while Cardano L1 remained the layer for lower-frequency, higher-assurance issuance of carbon assets. This is a sound design choice for a microgrid context, because it prevents the main chain from carrying the full burden of operational accounting.

The team also established:

- a working Hydra node setup using Dockerized infrastructure
  reserve wallet and operational wallet configuration
- token minting flows for internal accounting assets
- successful Hydra head initialization
- successful asset commits into the Hydra head
- successful transaction execution within the open head
- successful decommit approval and decommit finalization back to Layer 1

The frontend and local application environment were also sufficient to support asset minting and operator interaction during the PoC stage.

From an architecture perspective, the project correctly separated:

- frequent operational settlement
- deterministic energy accumulation
- periodic carbon issuance
- certificate-style Layer 1 anchoring

That separation is one of the strongest parts of the design.

Verified Technical Outcomes

Based on the PoC material shared, the following outcomes were clearly demonstrated:

- Hydra runtime environment was successfully configured using Hydra Node, etcd, Docker, and Cardano node infrastructure.
- Cardano node synchronization and wallet setup were completed successfully.
- Internal accounting assets were minted and assigned to reserve wallet UTxOs.
- Hydra head lifecycle events were validated, including:
  - `Init`
  - `Committed`
  - `HeadIsOpen`
  - in-head transaction execution
  - snapshot confirmation
  - decommit approval

    - decommit finalization

- UTxO transitions for committed and decommitted assets were explicitly shown and tracked.
- The PoC demonstrated the feasibility of keeping frequent microgrid accounting inside Hydra while using Cardano Layer 1 for more durable asset issuance and anchoring.

What has not yet been fully demonstrated, and should not be overstated, is pilot-grade operational readiness. The current implementation proves architectural feasibility, but not yet production resilience.

6.8 ClimaFi

Technical Engagement

Technical engagement with ClimaFi focused on defining a **verification-first architecture**, where real-world data validation acts as the primary trigger for any blockchain interaction.

Support sessions were centered around:

- Designing the **data ingestion and normalization pipeline** for heterogeneous meteorological and climate datasets (e.g., CSV, BUFR, API feeds)
- Reviewing the **verification engine logic**, including:
    - timeliness checks
    - consistency and range validation
    - completeness against predefined schemas (e.g., GBON/WIS2 or project-specific MRV criteria)
- Defining the **"Verification Gate" pattern**:
    - Pass → trigger tokenization / on-chain action
    - Fail → reject and log for audit
- Aligning **tokenization triggers** with verified data events rather than manual or API-driven minting
- Reviewing integration patterns between:
    - off-chain verification layer
    - backend orchestration services
    - on-chain minting policies and metadata standards

Additional support included discussions on **"shadow payment" architecture**, where verified data events simulate financial outcomes without executing real on-chain transfers, enabling regulatory-safe pilot deployments.

Key Technical Challenges

Several challenges emerged during the integration:

1. Standardizing heterogeneous data inputs
Climate and meteorological data sources vary significantly in format, frequency, and reliability, requiring normalization before verification.

2. Designing deterministic verification logic
Ensuring that identical datasets always produce identical validation results (critical for reproducibility and trust in results-based finance models).

3. Preventing invalid or low-quality data from triggering on-chain actions
Without a strict verification layer, token minting or financial flows could be triggered by incomplete or manipulated datasets.

4. Handling latency and intermittent connectivity
Real-world data pipelines (especially in developing regions) introduce delays that must be accounted for without penalizing valid data.

5. Linking off-chain datasets to on-chain state
Ensuring a **tamper-proof relationship** between verified data and blockchain records without exposing sensitive raw data on-chain.

Solutions Implemented

The ClimaFi architecture introduces a structured **Verification → Tokenization pipeline**:

Verification Engine (Off-chain):

- Rules-based validation system enforcing:
  - timeliness thresholds
  - schema validation
  - anomaly detection
- Deterministic processing to ensure reproducible results across identical datasets
- Rejection and logging of non-compliant data

Verification Gate Pattern:

- Explicit control point:
  - **Valid data → eligible for tokenization**
  - **Invalid data → blocked from on-chain interaction**

Tokenization Trigger Logic:

- Verified datasets trigger:
  - minting of **Data Receipt Tokens** or **Impact Tokens**
- Token metadata includes:

    - cryptographic hash of dataset
    - timestamp
    - source identifier

On-chain Integration:

- Cardano used for:
    - token minting
    - immutable metadata anchoring
- Plutus-based logic supports:
    - “shadow escrow” accounting models
    - state updates based on verified token presence

Shadow Accounting Layer:

- Simulated financial logic calculates:
    - theoretical payouts based on verified datasets
- Enables:
    - results-based financing validation
    - regulatory-safe pilot execution

Verified Technical Outcomes

Based on the PoC documentation and testing:

- End-to-end pipeline validated:
    - data ingestion → verification → token generation
- Verification engine successfully:
    - rejects invalid or incomplete datasets
    - produces consistent outputs for identical inputs
- Token minting confirmed:
    - only triggered by verified datasets
    - metadata correctly includes dataset hash and identifiers
- Dashboard layer demonstrates:
    - real-time visibility into verified vs rejected data
    - simulated earnings through “shadow payment” logic
- System demonstrates feasibility of:
    - linking real-world climate data to blockchain-based accounting
    - enabling **results-based finance without live payments**

7. Technical Integration Methodology

Cohort 2 projects followed a structured and repeatable integration methodology consistent with Cohort 1, while allowing flexibility for mature platforms and domain-specific requirements.

7.1 Smart Contract Development and Deployment

1. **Repository Onboarding**
   Teams were onboarded to EMURGO reference repositories or aligned custom contract structures based on their use case maturity.

2. **Contract Authoring (Aiken / Plutus)**

   - Authorization logic (admin, verifier, beneficiary roles)
   - State transitions (registration, verification, minting, retirement)
   - Time-locks and recovery logic where applicable

3. **Testing and Validation**

   - Unit testing using `aiken test`
   - Emulator-based integration tests
   - Explicit edge-case validation (unauthorized actions, duplicate minting, expired claims)

4. **Testnet Deployment**

   - Deployment to Cardano Preview or Preprod testnets
   - Verification of execution units and memory usage
   - Transaction hash documentation for auditability

7.2 Backend Integration

Backend services acted as the orchestration layer between user interfaces, external systems, and on-chain logic.

Key responsibilities included:

- API exposure for minting, burning, and verification actions
- Identity and role-based access control (JWT, API keys)
- Integration with external services (SMS, MRV inputs, document storage)
- Idempotency handling to prevent double execution

Backends were primarily implemented using:

- `template-backend-nestjs`

- Custom Node.js / Express services for specialized workflows

7.3 Frontend and Access Layers

Cohort 2 emphasized **multiple access patterns**, depending on the target user group:

User Type	Access Pattern	Example Projects
Beneficiaries (unbanked)	SMS / USSD	xCapit, Plastiks
Merchants / Operators	Web dashboards	xCapit
Regulators / Auditors	Read-only dashboards	Fuel Switch, TradeChainX
Public verifiers	QR-based verification	SALA
Farmers / Aggregators	Token-based traceability UI	Green Giraffe

This approach ensured usability across low-connectivity, institutional, and public contexts.

8. Cross-Project Architecture Patterns Observed

Several reusable architectural patterns emerged across Cohort 2:

8.1 Mint–Anchor–Burn Lifecycle

Used by:

- Fuel Switch (RECs)
- TradeChainX (carbon credits)

- ClimaFi (impact tokens)

This pattern ensures:

- Verifiable issuance
- One-time usage
- Prevention of double counting

8.2 Role-Based Smart Contract Enforcement

Used by:

- xCapit (Admin / Beneficiary / Steward)
- TradeChainX (Project Owner / Verifier)
- SALA (Issuer / Verifier)

Smart contracts enforce **who can act**, not just **what action is allowed**.

8.3 On-chain Integrity, Off-chain Privacy

Common across:

- SALA
- TradeChainX
- Green Giraffe

Sensitive data stored off-chain, with cryptographic hashes anchored on-chain to ensure integrity without exposing personal or confidential information.

9. Lessons Learned and Recommendations

9.1 Key Lessons

- **Architecture matters more than code reuse** for mature teams.
- Low-tech access (SMS / USSD) is essential for inclusion-focused SDG solutions.
- Identity and authorization logic should be **contract-enforced**, not backend-only.
- Aiken significantly reduces complexity for expressing real-world constraints.

9.2 Recommendations for Future Cohorts

Area	Recommendation
Identity	Provide prebuilt DID / credential scaffolds
Payments & Aid	Package LTW / USSD patterns as reusable modules
MRV	Introduce oracle-ready templates for automated data ingestion
Monitoring	Add standardized analytics dashboards for UNDP reporting
Security	Mandate external audits for production-bound pilots